Blob Storage Patterns

tier by access, protect by policy

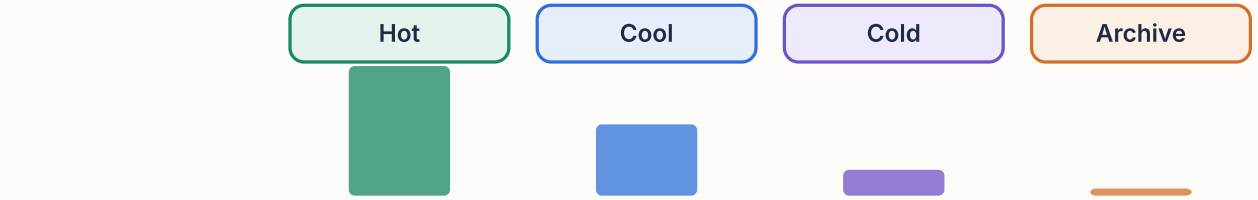
Watch the 4-min video

In one line: put each blob in the tier that matches how often it is *read* (hot, cool, cold, archive) and let lifecycle rules move it there by age; protect it with versioning and soft delete against mistakes, and with immutability (WORM) and legal holds against tampering.

1 The bill for data nobody reads

Every log file, export and backup goes into blob storage. The first month costs almost nothing. Two years later blob storage is one of the biggest lines on the cloud bill, and access logs show that almost nothing older than 30 days is ever read. Everything still sits in the hot tier, priced for data that is read every day. Typical data has a steep access curve: read heavily for days, occasionally for weeks, then almost never. Tiers and lifecycle rules make the price follow that curve.

2 Access tiers



	Hot	Cool	Cold	Archive
storage $/GB-month	≈ 0.018	≈ 0.010	≈ 0.0036	≈ 0.001
first byte	ms	ms	ms	hours
minimum duration	none	30 days	90 days	180 days
per-GB read cost	lowest	higher	higher still	rehydrate first

colder → cheaper to keep, dearer and slower to read

Figure 1. Azure Blob tiers (approximate East US LRS list prices). Storage gets cheaper as you go colder; reads get slower and dearer, and each tier has a minimum billable duration.

- **Hot**: highest storage price, cheapest reads and writes. Data in active use.
- **Cool** (≥ 30 days) and **Cold** (≥ 90 days): still **online** (millisecond reads) but each read and write costs more, plus a per-GB retrieval charge. Moving or deleting a blob earlier triggers an **early-deletion** charge for the remaining days.
- **Archive** (≥ 180 days): **offline**. You can't read the blob until you **rehydrate** it by changing its tier (or copying it) to an online tier: up to 15 hours at standard priority, often under an hour at high priority for blobs under 10 GB, at a higher price. Archive is only available on LRS/GRS-type accounts, not zone-redundant ones.

Amazon S3 has the same ladder as storage classes: Standard → Standard-IA (30 days) → Glacier Instant Retrieval (90) → Glacier Flexible Retrieval (90, restore in minutes to hours) → Glacier Deep Archive (180, restore within 12 hours). Google Cloud Storage: Standard, Nearline (30), Coldline (90), Archive (365).

3 The trade-off in numbers

The right tier depends on **how often data is read**, not on how important it is. A compliance archive read once a year belongs in archive even though it matters a lot; a small config file read a million times a day belongs in hot.

```
cost per month = GB stored × price + operations + GB read × retrieval
```

Moving 1 GB from hot to cool saves about $0.008 per month, while reading it from cool costs about $0.01 in retrieval. So if data is read roughly **once a month or more**, cool costs *more* than hot. Two more traps: tiering many **tiny objects** (each transition is a billed write operation, and S3's IA classes bill a minimum of 128 KB per object), and the **minimum durations**: data deleted after 40 days should never go to archive. When access patterns are unknown, let the platform decide: S3 **Intelligent-Tiering** moves objects between tiers based on observed access (small per-object monitoring fee), and Azure lifecycle rules can use `daysAfterLastAccessTimeGreaterThan` once last-access tracking is enabled.

4 Protect against mistakes: versioning and soft delete

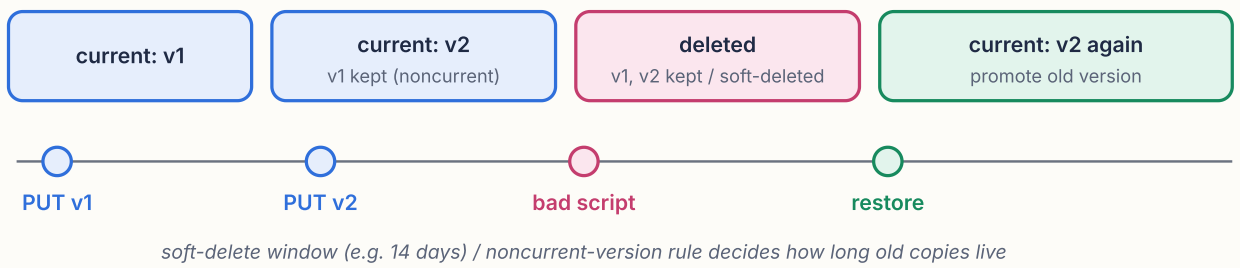


Figure 2. With versioning, an overwrite or delete never destroys data; the old version is promoted back. Old versions are billed until a rule removes them.

- **Versioning** (Azure Blob versioning, S3 Versioning) keeps every previous version automatically. In S3 a delete adds a *delete marker*; removing the marker brings the object back.
- **Soft delete** (Azure) keeps deleted blobs, and with container soft delete whole containers, for a retention window of 1–365 days. Add a resource lock so nobody can delete the storage account itself.
- Both multiply storage: a file overwritten daily keeps 30 versions a month. Pair them with a lifecycle rule that deletes noncurrent versions after N days (lesson 12).
- They don't stop a **privileged** attacker, who can disable soft delete or purge versions. That's what immutability is for.

5 Data that must never change: WORM and legal holds

Immutability (write once, read many) locks blobs so they can't be overwritten or deleted until a retention period ends, not even by an administrator or the account owner.

	Azure immutable storage	S3 Object Lock
Time-based retention	policy on container or version; *unlocked* for testing, *locked* = can only be extended	**governance** mode: users with a special permission can bypass; **compliance** mode: nobody can shorten or delete, not even root
Legal hold	tag-based hold, no expiry, until cleared	on/off flag per object version, independent of retention
Prerequisite	version-level WORM needs versioning	versioning must be on
Compliance	both have been assessed for SEC 17a-4(f), FINRA and CFTC record-keeping rules	

Uses: audit logs, financial records, and **backups that ransomware can't encrypt or delete** (lesson 13). Test with governance mode or an unlocked policy first: a locked compliance-mode retention of 7 years on the wrong data can't be undone, and you pay for it for 7 years.

6 Naming keys to avoid hot partitions

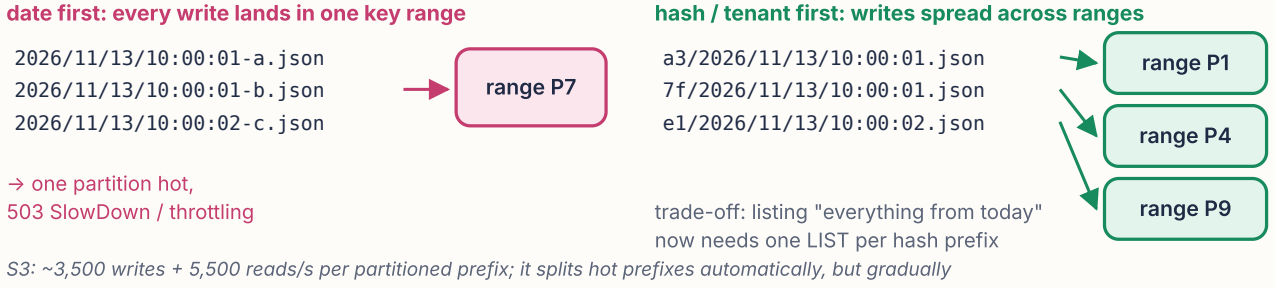


Figure 3. Object stores partition the key space by name ranges. Keys that start with a timestamp send all new writes to one range.

S3 supports at least 3,500 PUT/COPY/POST/DELETE and 5,500 GET/HEAD requests per second **per partitioned prefix**, and splits busy prefixes automatically; during the ramp you may see `503 Slow Down`, so clients must retry with exponential backoff. Azure partitions by account + container + blob name, also range-based, and documents per-blob (≈ 500 requests/s) and per-account request targets. Most workloads never notice. At very high request rates, put a short hash or tenant ID *before* the date (`7f/2026/11/13/…`) so writes spread, or spread load across several prefixes or accounts.

7 Example: lifecycle for logs

```
{ "rules": [ { "name": "logs-tiering", "enabled": true, "type": "Lifecycle",
    "definition": {
      "filters": { "blobTypes": ["blockBlob"],
                   "prefixMatch": ["appdata/logs/"] },   # container/prefix
      "actions": { "baseBlob": {
        "tierToCool":    { "daysAfterModificationGreaterThan": 30 },
        "tierToArchive": { "daysAfterModificationGreaterThan": 180 },
        "delete":        { "daysAfterModificationGreaterThan": 365 } } } } ] }
```

Figure 4. Azure Blob lifecycle policy. `prefixMatch` starts with the container name; the policy runs about once a day.

WORKED EXAMPLE: LOGS TO COOL AFTER 30 DAYS, ARCHIVE AFTER 180, DELETE AFTER 365

A platform writes **10 TB of logs a month** and keeps them for a year, so about 120 TB is stored at any time. All in hot at ≈ $0.018/GB: 120,000 GB × 0.018 ≈ **$2,160/month**. With the policy: the newest 10 TB in hot ($180), 50 TB aged 30–180 days in cool (× 0.010 = $500), 60 TB aged 180–365 days in archive (× 0.001 ≈ $60): **≈ $740/month**, about 65% less, plus small transition charges.

The minimum durations fit: blobs spend 150 days in cool (≥ 30) and 185 days in archive (≥ 180). Had the policy deleted logs at day 300, every blob would pay an early-deletion charge for its last 60 archive days. And if incident investigations sometimes need 3-month-old logs, keep them online in cool or cold rather than archive: a 15-hour rehydration during an outage is not acceptable.

TRADE-OFFS

- Colder tiers: lower storage price, higher per-read cost, minimum durations, slower (archive: hours) access.
- Versioning and soft delete: cheap insurance against mistakes, but they grow storage until a rule prunes old versions.
- WORM: protects against insiders and ransomware, but also against you; legitimate deletion (e.g. a GDPR request) waits for retention to end.
- Hashed key prefixes: spread load, but make time-range listing and browsing harder.

COMMON MISTAKES

- Choosing tiers by importance instead of read frequency.
- Archiving data you may need quickly (incident logs, restores with a tight RTO).
- Tiering millions of tiny objects, where transition and minimum-size charges exceed the savings.
- Enabling versioning without a noncurrent-version rule, then wondering why storage doubled.

8 Key terms

Access tier / storage class: price-vs-access option: hot, cool, cold, archive

Early deletion charge: fee for leaving a tier before its minimum duration

WORM / immutability: no overwrite or delete until retention ends

Rehydration: bringing an archived blob back online before reading it

Soft delete: deleted data kept for a retention window, restorable

Legal hold: open-ended lock for litigation or investigation

9 Interview questions

Q1. How do blob tiers reduce cost? BASIC

Rarely read data moves to cooler, cheaper tiers (cool, cold, archive), automatically by age via lifecycle rules, trading slower and costlier access for much lower storage prices.

Q2. How do you protect blobs from mistakes and tampering? BASIC

Versioning and soft delete undo accidental overwrites and deletes; immutability policies (WORM) and legal holds lock data that must never change, even for admins.

Q3. A team asks to move all data older than 7 days to archive to save money. What do you check first? SCENARIO

How often old data is read and how fast it's needed (archive takes hours to rehydrate), object sizes and counts (transition costs), and how long it's kept (180-day minimum).

Q4. Ransomware gets admin credentials to your storage account. What would have saved your backups? SCENARIO

Immutable (locked WORM / compliance-mode Object Lock) copies, ideally replicated to a separate account; soft delete and versioning alone can be disabled by an admin.

Q5. Your service writes 20,000 objects/s with keys starting with a timestamp and sees 503 errors. Fix? SCENARIO

Retry with backoff, and change the key scheme to put a hash or tenant prefix first so writes spread across partitions; keep a time index elsewhere for listing.

Q6. When is the cool tier more expensive than hot? TRICKY

When data is read often, roughly once a month or more per byte, since retrieval and operation charges outweigh the storage saving; also when data is deleted before 30 days.

Q7. Governance vs compliance mode in S3 Object Lock? TRICKY

Governance can be bypassed by users granted a special permission (useful for testing); compliance can't be shortened or removed by anyone, including the root account, until retention ends.

Next: SD-O-004 · File Upload Architecture backendroadmap.com